

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: MLA03

COURSE CODE: Reinforcement learning

COURSE OUTCOME COVERED

CO1: Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems. (BL2, BL3)

Assessment Tool Weightage: 50%

Dr. G. A. Senthil

Code of Conduct:

I am **A.Shiva Shankar Reddy/192425174** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Shiva Shankar Reddy

QUESTIONS: 10

Total Marks: 50

Annexure A

RL Basic Experiments demo with Keras and TensorFlow using Anaconda navigator

Duration: 2hrs

Q.2Aim: To create, initialize, and interact with standard Reinforcement Learning environments such as FrozenLake, CartPole, and MountainCar using Gymnasium to understand the concepts of states, actions, rewards, and episodes.

Procedure

1. Import Gymnasium.
2. Create FrozenLake environment.
3. Reset environment.
4. Perform random actions.
5. Observe state and reward.

Code

Python

Run

```
import gymnasium as gym
```

```
env = gym.make("FrozenLake-v1")
```

```
state, info = env.reset()
```

```
for i in range(5):
```

```
    action = env.action_space.sample()
```

```
    state, reward, terminated, truncated, info = env.step(action)
```

```
    print("State:", state, "Reward:", reward)
```

```
env.close()
```

Output:

	Console
<pre>import gymnasium as gym env = gym.make("FrozenLake-v1") state, info = env.reset() for i in range(5): action = env.action_space.sample() state, reward, terminated, truncated, info = env.step(action) print("State:", state, "Reward:", reward) env.close()</pre>	<pre>Run started Initializing environment Installing packages Running code State: 0 Reward: 0 State: 1 Reward: 0 State: 2 Reward: 0 State: 1 Reward: 0 State: 1 Reward: 0 Run completed in 6603ms</pre>

Result

Successfully interacted with the FrozenLake environment and observed states and rewards.

Q.3.Aim: To implement the basic Reinforcement Learning framework by designing an agent that interacts with an environment through states, actions, rewards, and policies using Python.

Algorithm

1. Initialize environment.
2. Reset environment.
3. Select an action.
4. Execute action.
5. Receive reward.
6. Repeat until episode ends.

Code

Python

Run

```
import gymnasium as gym
```

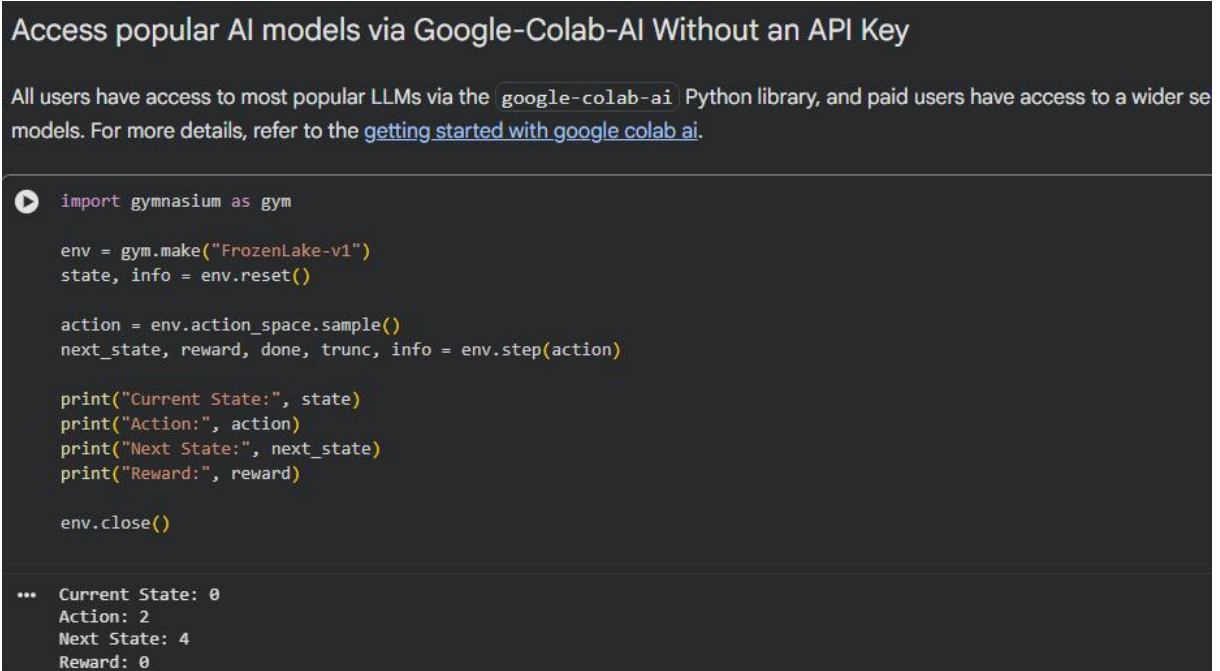
```
env = gym.make("FrozenLake-v1")
state, info = env.reset()
```

```
action = env.action_space.sample()
next_state, reward, done, trunc, info = env.step(action)
```

```
print("Current State:", state)
print("Action:", action)
print("Next State:", next_state)
print("Reward:", reward)
```

```
env.close()
```

OUTPUT:



Access popular AI models via Google-Colab-AI Without an API Key

All users have access to most popular LLMs via the `google-colab-ai` Python library, and paid users have access to a wider set of models. For more details, refer to the [getting started with google colab ai](#).

```
import gymnasium as gym

env = gym.make("FrozenLake-v1")
state, info = env.reset()

action = env.action_space.sample()
next_state, reward, done, trunc, info = env.step(action)

print("Current State:", state)
print("Action:", action)
print("Next State:", next_state)
print("Reward:", reward)

env.close()
```

```
... Current State: 0
Action: 2
Next State: 4
Reward: 0
```

Result: The RL framework was implemented successfully.

Q.4.Aim: To develop a simple Markov Decision Process model and simulate state transitions, reward functions, and policy execution for understanding sequential decision-making problems.

Algorithm

1. Define states.
2. Define actions.
3. Define rewards.
4. Select action.
5. Move to next state.
6. Receive reward.

Code

Python

Run

```
states = ["S1", "S2", "S3"]
```

```
current_state = "S1"
```

```
if current_state == "S1":  
    next_state = "S2"  
    reward = 10
```

```
print("Current State:", current_state)  
print("Next State:", next_state)  
print("Reward:", reward)
```

OUTPUT:



The screenshot shows a Jupyter Notebook cell with the following Python code:

```
[3] ✓ 0s states = ["S1", "S2", "S3"]  
  
current_state = "S1"  
  
if current_state == "S1":  
    next_state = "S2"  
    reward = 10  
|  
print("Current State:", current_state)  
print("Next State:", next_state)  
print("Reward:", reward)
```

The output of the code is displayed below the code cell:

```
... Current State: S1  
Next State: S2  
Reward: 10
```

Result

MDP state transition and reward simulation were performed successfully.

Q.6.Aim: To implement ϵ -Greedy, Greedy, and Random action-selection strategies and compare their performance in balancing exploration and exploitation during Reinforcement Learning.

Algorithm

1. Define rewards for actions.
2. Set ϵ (epsilon) value.
3. Generate a random number.
4. If random number $< \epsilon$, choose a random action (Exploration).
5. Otherwise choose the best action (Exploitation).
6. Display selected action.

Code

Python

Run

```
import random
```

```
rewards = [10, 20, 15, 30]
```

```
epsilon = 0.2
```

```
if random.random() < epsilon:
```

```
    action = random.randint(0, 3)
```

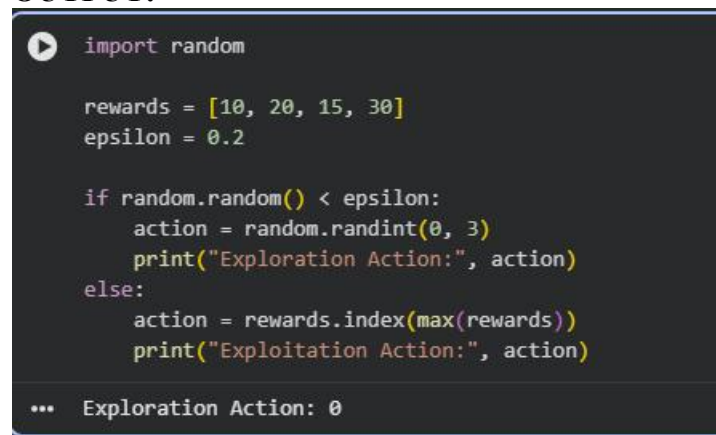
```
    print("Exploration Action:", action)
```

```
else:
```

```
    action = rewards.index(max(rewards))
```

```
    print("Exploitation Action:", action)
```

OUTPUT:



```
import random

rewards = [10, 20, 15, 30]
epsilon = 0.2

if random.random() < epsilon:
    action = random.randint(0, 3)
    print("Exploration Action:", action)
else:
    action = rewards.index(max(rewards))
    print("Exploitation Action:", action)
```

... Exploration Action: 0

Result

The ϵ -Greedy strategy was implemented successfully to balance exploration and exploitation.

Q.8Aim: To simulate an RL agent in a simple environment and visualize cumulative rewards, episode rewards, and learning performance using Matplotlib graphs.

Procedure

1. Import Matplotlib.
2. Create reward data.
3. Plot rewards.
4. Display graph.

Code

Python

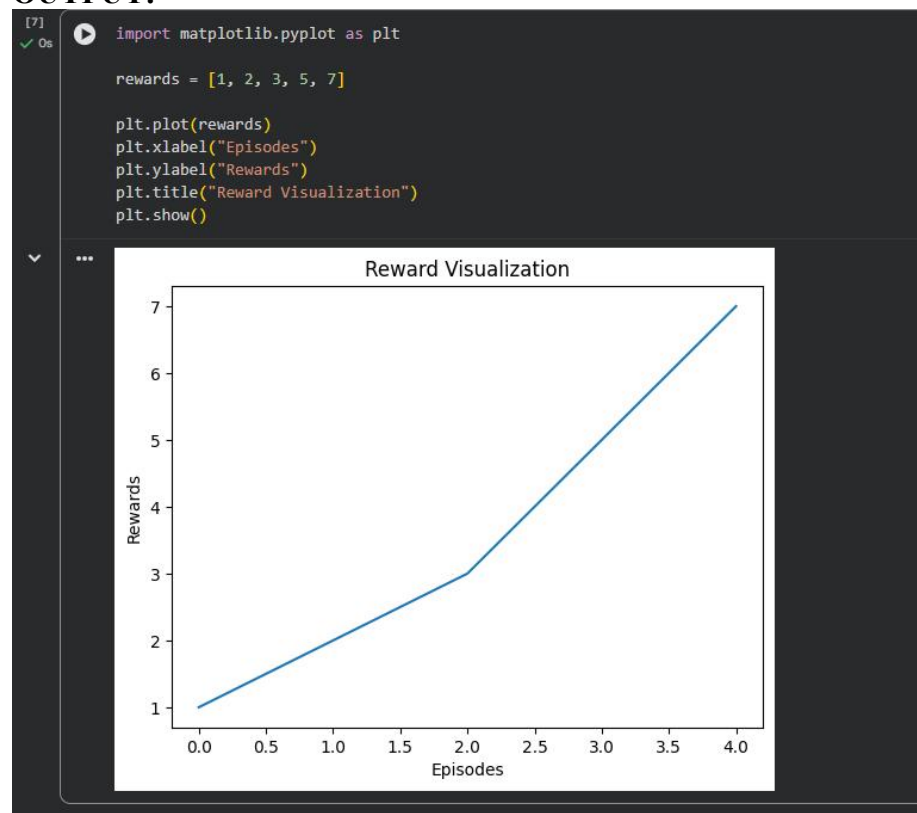
Run

```
import matplotlib.pyplot as plt
```

```
rewards = [1, 2, 3, 5, 7]
```

```
plt.plot(rewards)
plt.xlabel("Episodes")
plt.ylabel("Rewards")
plt.title("Reward Visualization")
plt.show()
```

OUTPUT:



Output

A graph showing rewards increasing over episodes.

Result

Reward visualization was generated successfully using Matplotlib

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand